

7th Meeting 12/9

1. Explain gradient 1 / loss function value.

KENDAL function using Uncertainty to Weigh Losses. Start with softmax

Gaussian → continuous number (coordinates, bounding box, depth)

Softmax → discrete categories (a vs b) (kendal is using for semantic segmentation) Write the mathematical equation function loss (explain the normal loss function)

weight: $\frac{1}{2\sigma^2}$

Uncertainty: σ

$$\begin{aligned}
 &= -\log p(\mathbf{y}_1, \mathbf{y}_2 = c | \mathbf{f}^{\mathbf{W}}(\mathbf{x})) \\
 &= -\log \mathcal{N}(\mathbf{y}_1; \mathbf{f}^{\mathbf{W}}(\mathbf{x}), \sigma_1^2) \cdot \text{Softmax}(\mathbf{y}_2 = c; \mathbf{f}^{\mathbf{W}}(\mathbf{x}), \sigma_2) \\
 &= \frac{1}{2\sigma_1^2} \|\mathbf{y}_1 - \mathbf{f}^{\mathbf{W}}(\mathbf{x})\|^2 + \log \sigma_1 - \log p(\mathbf{y}_2 = c | \mathbf{f}^{\mathbf{W}}(\mathbf{x}), \sigma_2) \\
 &= \frac{1}{2\sigma_1^2} \mathcal{L}_1(\mathbf{W}) + \frac{1}{\sigma_2^2} \mathcal{L}_2(\mathbf{W}) + \log \sigma_1 \\
 &\quad + \log \frac{\sum_{c'} \exp\left(\frac{1}{\sigma_2^2} f_{c'}^{\mathbf{W}}(\mathbf{x})\right)}{\left(\sum_{c'} \exp\left(f_{c'}^{\mathbf{W}}(\mathbf{x})\right)\right)^{\frac{1}{\sigma_2^2}}} \\
 &\approx \frac{1}{2\sigma_1^2} \mathcal{L}_1(\mathbf{W}) + \frac{1}{\sigma_2^2} \mathcal{L}_2(\mathbf{W}) + \log \sigma_1 + \log \sigma_2,
 \end{aligned} \tag{10}$$

where again we write $\mathcal{L}_1(\mathbf{W}) = \|\mathbf{y}_1 - \mathbf{f}^{\mathbf{W}}(\mathbf{x})\|^2$
for the Euclidean loss of $\mathbf{y}_1$, write $\mathcal{L}_2(\mathbf{W}) =$

But for my model, I use gaussian:

Function:

$$P(y|x, w) = \mathcal{N}(y; f(x), \sigma^2)$$

- **P**: "Probability"
- **I** : "Given" (conditional bar)

- **N** : "Normal" (referring to the distribution)
- **f(x)**: "f of x" (the mean/prediction)
- **σ²**: "Sigma squared" (the variance)

The full Gaussian Probability Density Function (PDF):

$$P(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\|y - f(x)\|^2}{2\sigma^2}\right)$$

To train a model, we want to Maximize Probability (Likelihood), which is mathematically the same as **Minimizing Negative Log-Likelihood (NLL)**.

$$NLL = -\log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\text{Loss}}{2\sigma^2}\right)\right)$$

The Expansion

Using log rules ($\log(a \cdot b) = \log a + \log b$):

$$NLL = -\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \log\left(\exp\left(-\frac{\text{Loss}}{2\sigma^2}\right)\right)$$

Simplify the exponential ($\log(\exp(x)) = x$)

$$NLL = -\log\left((2\pi\sigma^2)^{-1/2}\right) - \left(-\frac{\text{Loss}}{2\sigma^2}\right)$$

Simplify the first term

$$NLL = \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \text{Loss}$$

expand the log

$$\log(2\pi\sigma^2) = \log(2\pi) + \log(\sigma^2) = \text{Constant} + 2 \log \sigma$$

$$NLL = \underbrace{\frac{1}{2\sigma^2} \text{Loss}}_{\text{Weighted Error}} + \underbrace{\log \sigma}_{\text{Regularizer}} + \underbrace{\text{Constant}}_{\text{Ignored}}$$

$$\mathcal{L}_{total} = \underbrace{\frac{1}{2\sigma_{det}^2} \mathcal{L}_{det}}_{Detection} + \underbrace{\frac{1}{2\sigma_{pose}^2} \mathcal{L}_{pose}}_{Pose} + \underbrace{\frac{1}{2\sigma_{act}^2} \mathcal{L}_{act}}_{Activity} + \underbrace{\log(\sigma_{det}) + \log(\sigma_{pose}) + \log(\sigma_{act})}_{Regularization}$$

Gradient function:

$$J(\theta) = -\ln(P(\theta))$$

$$\frac{dJ}{d\theta} = \frac{dJ}{dP} \times \frac{dP}{d\theta}$$

$$\frac{dJ}{dP} = \frac{d}{dP}[-\ln(P)] = -\frac{1}{P}$$

$$\nabla J = \left(-\frac{1}{P}\right) \times \nabla P$$

$$\text{Gradient} \propto \frac{1}{P}$$

Loss is proportional to Probability

$$Loss(L) = -\ln(P)$$

example:

1. Perfect prediction

$$P = 1.0 : L = -\ln(1) = 0$$

2. Bad prediction

$$P = 0.1 : L = -\ln(0.1) \approx 2.3$$

3. Terrible prediction

$$P = 0.001 : L = -\ln(0.001) \approx 6.9$$

The explanation without weighted loss function

Pose Loss is **0.0005**

$$\text{Gradient} \propto \frac{1}{\text{Loss}}$$

LR: 0.001

$$\text{Gradient} = \frac{1}{0.0005} = \frac{1}{5 \times 10^{-4}} = \mathbf{2,000.0}$$

Weight Update

$$\text{Update} = \text{Gradient} \times \text{LR} = 2,000.0 \times 0.001 = \mathbf{2.0}$$

update model weight

$$W_{\text{new}} = W_{\text{old}} - \text{Update} = 0.01 - 2.0 = \mathbf{-1.99}$$

Forward Pass

$$Z = W_{\text{new}} \times \text{Input} = -1.99 \times 500 \approx \mathbf{-1,000}$$

There will be function failure

$$\text{Value} = e^{-Z} = e^{1000}$$

$$e^{1000} \approx \text{Infinity}$$

$$\text{Result} = \frac{\infty}{\infty} = \text{NaN}$$

Conclusion: The tiny number **0.0005** started a chain reaction that resulted in **Infinity**, crashing the training.

Softmax Formula:

$$P_i = \frac{e^{Z_i}}{\sum_j e^{Z_j}}$$

$$\text{Result} = \frac{\text{Numerator}}{\text{Denominator}} = \frac{\infty}{\infty}$$

What if I use Kendall loss?

$$\text{Weight} = \frac{1}{2\sigma^2} = \frac{1}{2(100)^2} = \frac{1}{20,000} = \mathbf{0.00005}$$

Why the sigma is 100?, assume the total gradient (0.1 [safe])

$$\text{Total Gradient} = \frac{1}{2\sigma^2} \times \text{Raw Gradient}$$

$$0.1 = \frac{1}{2\sigma^2} \times 2000$$

$$0.2\sigma^2 = 2000$$

$$\sigma^2 = \frac{2000}{0.2} = 10,000$$

$$\sigma = \sqrt{10,000} = \mathbf{100}$$

Back to topic:

$$\text{Scaled Gradient} = \text{Weight} \times \text{Raw Gradient}$$

$$\text{Scaled Gradient} = 0.00005 \times 2,000.0 = \mathbf{0.1}$$

$$\text{Update} = \text{Scaled Gradient} \times \text{LR} = 0.1 \times 0.001 = \mathbf{0.0001}$$

$$W_{\text{new}} = 0.01 - 0.0001 = \mathbf{0.0099}$$

How did they adjust the value of this gradient (in the paper)

$$L(\sigma) = \frac{1}{2\sigma^2} \text{Loss}_{\text{raw}} + \log(\sigma)$$

term 1

$$\frac{d}{d\sigma} \left(\frac{1}{2}\sigma^{-2} \cdot \text{Loss} \right) = \frac{1}{2}(-2\sigma^{-3}) \cdot \text{Loss} = -\frac{\text{Loss}}{\sigma^3}$$

term 2

$$\frac{d}{d\sigma}(\log \sigma) = \frac{1}{\sigma}$$

so

$$\frac{\partial L}{\partial \sigma} = -\frac{\text{Loss}}{\sigma^3} + \frac{1}{\sigma}$$

$$\sigma_{new} = \sigma_{old} - lr \times \left(\frac{1}{\sigma} - \frac{\text{Loss}}{\sigma^3} \right)$$

- **Scenario A: High Error (Loss is Big, 100)**

- We assume current

$$\sigma = 1$$

- **Update:**

$$\nabla_{\sigma} = \frac{1}{1} - \frac{100}{1^3} = 1 - 100 = -99$$

- Gradient $\sim 1 - 100 = -99$ (Negative).

$$\sigma_{new} = 1 - (-0.01 \times -99) = 1 + 0.99 \approx 2$$

- **Result:** Sigma **INCREASES**.
- **Why?** The model says "The error is huge! I need to increase uncertainty (σ) to lower the weight and reduce the penalty!"

- **Scenario B: Low Error (Loss is Small, 0.1)**

- We assume current $\sigma = 1$
- Gradient $\approx 1 - 0.1 = +0.9$ (Positive).
- **Update:** $\sigma_{new} = 1 - (0.01 \times 0.9) = 0.991$

- **Result:** Sigma **DECREASES**.

- **Why?** The model says "The error is tiny! I can afford to lower uncertainty (σ) to decrease the regularization penalty ($\log \sigma$)."

2. Write the mathematical equation function loss (explain the normal loss function)

- **A. Detection Loss** ($\mathcal{L}_{det}$)

$$\mathcal{L}_{det} = \mathcal{L}_{box} + \mathcal{L}_{obj} + \mathcal{L}_{cls}$$

$$\mathcal{L}_{box} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2}$$

- $\mathcal{L}_{box} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2}$

- IoU: Intersection over Union (Overlap Area / Union Area).
- $\rho^2(b, b^{gt})$: Euclidean distance between the center point of the predicted box (b) and the ground truth box (b^{gt}).
- c^2 : The diagonal length of the smallest enclosing box covering both boxes.

- **How it is calculated (Step-by-Step):**

1. **Coordinates:** Convert predictions and targets to corners (x_1, y_1, x_2, y_2).
2. **Intersection:** Calculate the area of overlap.
3. **Union:** $Area_{pred} + Area_{target} - Area_{overlap}$.
4. **IoU:** $\frac{Intersection}{Union}$.
5. **Center Distance:** Calculate $(x_{center_pred} - x_{center_target})^2 + (y_{center_pred} - y_{center_target})^2$.
6. **Enclosing Box:** Find the smallest box that contains both the prediction and the target. Calculate its diagonal squared (c^2).
7. **DIoU Term:** Subtract $\frac{CenterDistance}{Diagonal^2}$ from the IoU.
8. **Final Loss:** $1.0 - (IoU - Penalty)$.

- B Objectness Loss ($\mathcal{L}_{obj}$)

This asks: "Is there an object in this anchor box?"

- **Code Reference:** `self.bce(pred_obj, tgt_obj)`

- **The Math (Binary Cross Entropy with Logits):** $\mathcal{L}_{obj} = -[y \cdot \log(\sigma(x)) + (1 - y) \cdot \log(1 - \sigma(x))]$
 - x : The raw output from the network (logit).
 - $\sigma(x)$: The Sigmoid function $\frac{1}{1+e^{-x}}$ (converts logit to 0-1 probability).
 - y : The target (1.0 if object exists, 0.0 if background).
- C. Classification Loss ($\mathcal{L}_{cls}$)

This asks: "Given there is an object, is it a Person, Bottle, or Barcode?"

- **Code Reference:** `self.bce(pred_cls, tgt_cls_onehot)`
- **The Math:** Same BCE formula as above, but calculated for each class channel.

B. Pose Loss ($\mathcal{L}_{pose}$)

$$\mathcal{L}_{pose} = \mathcal{L}_{kpt} + 0.5 \times \mathcal{L}_{vis}$$

1. Keypoint Regression Loss (Smooth L1)

Smooth L1 is a regression loss that is robust to outliers.

- **Code Reference:** `F.smooth_l1_loss(pred_xy[visible_mask], ...)`
- **The Math:** $\mathcal{L}_{kpt}(x, y) = \begin{cases} 0.5(x - y)^2, & \text{if } |x - y| < 1 \\ |x - y| - 0.5, & \text{otherwise} \end{cases}$
 - **Interpretation:** If the error is small (<1), it behaves like **MSE** (Squared Error) to be precise. If the error is huge (>1, the model guesses the hand is on the ceiling), it behaves like **L1** (Absolute Error) to prevent massive gradients from exploding the training.
- **How it is calculated:**
 1. **Masking:** The code calculates `visible_mask = tgt_vis > 0.5`. It *only* learns from keypoints that actually exist in the image (if the legs are cut off, it doesn't penalize the model for not finding them).
 2. **Difference:** Calculate Prediction - Target.
 3. **Apply Function:** Apply the Smooth L1 formula to these differences.

Note:

- Strict regularization L1 regularization →
- For smaller $|x-y|$
- For larger →
- I will find the reason → source of the contribution → compare the model to this loss function
- Find the contribution relating to the loss function → I need to find the doubt

2. Visibility Loss ($\mathcal{L}_{vis}$)

This asks: "Is this keypoint visible in the image?"

- **Code Reference:** `bce(pred_vis, tgt_vis)`
- **The Math:** Binary Cross Entropy (same as Objectness above).
 - It trains a specific neuron to output `1.0` if the joint is visible and `0.0` if it is occluded/missing.

C. Activity Loss ($\mathcal{L}_{act}$)

$$\mathcal{L}_{act} = \text{CrossEntropy}(\text{Predictions}, \text{Targets})$$

1. The Math (Softmax + NLL)

PyTorch's `CrossEntropyLoss` combines two steps into one for numerical stability.

1. **Softmax (Get Probabilities):** $P(class_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

- z : The raw logits (scores) for Checking, Rotation, Storing.
- This forces all scores to sum to 1.0.

2. **Negative Log Likelihood (Calculate Error):** $\mathcal{L}_{act} = -\log(P(class_{true}))$

- It looks at the probability assigned to the *correct* class.
- If probability is 1.0, Loss is 0.

- If probability is low (0.1), Loss is high (2.3).
- **How it is calculated:**
 1. Take the raw output vector (`[2.0, -1.0, 0.5]`).
 2. Identify the target class index (Class 0 "Checking").
 3. Apply the Softmax-Log formula efficiently.

Summary of the "Real" Function Calculation

When you run `total_loss.backward()`, the chain rule flows through this exact hierarchy:

1. **Master Kendall Equation:** $\mathcal{L}_{total} = \frac{1}{2\sigma_{det}^2} \mathcal{L}_{det} + \frac{1}{2\sigma_{pose}^2} \mathcal{L}_{pose} + \frac{1}{2\sigma_{act}^2} \mathcal{L}_{act} + \sum \log(\sigma)$

2. **The Sub-Functions:**

- $\mathcal{L}_{det}$ comes from **DIoU** (Geometric overlap) and **BCE** (Probability).
- $\mathcal{L}_{pose}$ comes from **Smooth L1** (Robust regression distance).
- $\mathcal{L}_{act}$ comes from **Cross Entropy** (Categorical probability).

Smooth L1 prevents outlier explosions in Pose, DIoU stabilizes Box gradients, and Kendall Weighting balances the three tasks dynamically.

3. Architectural Breakdown

$$\text{YOLOv8} \xrightarrow{\text{Crop}} \text{OpenPose} \rightarrow \text{ResNet-Activity}$$

- The image is processed by three separate backbones → high latency, not optimal
- The Activity classifier only sees raw image pixels (or global features) after the crop → Low accuracy
- Loss gradients only update their own mode → Backbone features cannot be optimized globally for all three tasks

Workernet

Backbone → YOLOv8 → featured extracted at once → make the heavy duty to be more efficient

Neck → P3 (high-res) through P5 (high-context) → Ensures both small objects (barcodes) and large features (the person's torso) are visible to all heads

Parallel head

Detection → Bounding Boxes + Classes → YOLO-style Box/Class/Objectness prediction → P3, P4, P5 features

Pose → 13 Keypoints (x, y, visibility) → Predicts coordinates directly (Regression) → P3, P4, P5 features

Activity → Classification → **Intelligent Fusion (FiLM)** using P3, P4, and Pose → P3, P4 features + Pose vector

Mechanism 1: Pose Encoder (MLP)

The raw, normalized keypoint coordinates are processed into a dense context vector $\mathbf{z}$.

- **Input:** Flattened keypoints $\mathbf{K} \in \mathbb{R}^{13 \times 3} \rightarrow \mathbb{R}^{39}$
- **Math (Simplified):** $\mathbf{z}_{pose} = \text{MLP}(\mathbf{K}) \in \mathbb{R}^{64}$
 - **Logic:** Converts unstructured coordinate data into a dense, abstract "Pose Context" vector, focusing on relative joint positions.

```
self.pose_encoder = nn.Sequential(  
    nn.Linear(num_joints * 3, 128), # 39 → 128 (Expansion)  
    nn.LayerNorm(128),  
    nn.GELU(),  
    nn.Linear(128, 64), # 128 → 64 (Compression/Output)  
    nn.LayerNorm(64),  
    nn.GELU()  
)
```

Stage A: Initial Expansion (39 → 128)

The first layer dramatically increases the dimension from 39 to 128. This is where the network "explodes" the data into potential meaningful combinations.

- **Logic:** It gives the network 128 "slots" to store interpreted features like:
 - Slot 1: "Is the left arm fully extended?"
 - Slot 2: "Is the right wrist near the bottle?"
 - Slot 3: "Is the worker facing forward?"
 This expansion is necessary because the information content of 39 simple coordinates is far lower than the **semantic content** of the posture (the difference between x1 and x2 becomes a "difference feature" in the 128-dimensional space).

Stage B: Final Compression (128 → 64) - The Context Vector

The network then compresses this rich 128-dimensional representation down to 64.

- **Output Dimension (64):** Choosing 64 is a common engineering choice. It's a dense, concise **Pose Context Vector (zpose)** that is large enough to contain all the necessary postural information (Is the worker checking? Rotating? Storing?) but small enough to be efficiently concatenated with the image features later.

Mechanism 2: FiLM Modulation (Feature-wise Linear Modulation)

The Pose Context ($\mathbf{z}_{pose}$) dynamically adjusts the global CNN features (P3) before pooling.

- **Input:** P3 feature map ($\mathbf{F}_{P3}$) and Pose Context ($\mathbf{z}_{pose}$).
- **Math (Affine Transformation):** $\mathbf{F}'_{P3} = \gamma(\mathbf{z}_{pose}) \odot \mathbf{F}_{P3} + \beta(\mathbf{z}_{pose})$
 - $\gamma(\mathbf{z})$: Learned Scale factor (Multiplies the feature channels).
 - $\beta(\mathbf{z})$: Learned Bias factor (Adds to the feature channels).
 - **Logic:** If the pose encoder detects a "raised hand" pose, γ and β will turn up the signal for the "hand feature" channels in $\mathbf{F}_{P3}$ while suppressing irrelevant features. This is a powerful, lightweight form of **conditional attention**.

Scenario	Pose Context (zpose)	Generated γ and β	Modulated Feature (F')
Normal Frame	zpose is neutral	$\gamma \approx 1.0, \beta \approx 0.0$	Features are mostly unchanged (standard

			CNN output)
Action: "Checking"	zpose indicates Left Wrist Raised	yhand-channel=2.5 (Amplify this channel) βhand-channel=0.8 (Boost baseline)	The signal for "Left Hand" is now significantly stronger and easier to detect by the subsequent layers
Action: "Rotation"	zpose indicates Torso Bent/Twisted	ytorso-channel=3.0 (Amplify torso movement) βhand-channel=-0.5 (Suppress static hand features)	The features related to the torso and twisting motion are boosted, while less relevant hand features are suppressed

Mechanism 3: Pose-Guided Feature Sampling

The model explicitly samples high-resolution features from the neck (P4) at the exact pixel locations of the keypoints.

- **Input:** P4 feature map ($\mathbf{F}_{P4}$) and *KeypointCoordinates*(x_j, y_j)
- **Math (Bilinear Sampling):**

$$\mathbf{s}_j = \text{BilinearSample}(\mathbf{F}_{P4}, (x_j, y_j))$$

- **Logic:** This provides direct, high-fidelity visual evidence right where the action is (the texture/color of the elbow or wrist). This is crucial for distinguishing between 'Checking' (stationary hand) and 'Rotation' (rotating wrist) based on local visual cues.

Final fusion

$$\mathbf{y} = \text{MLP}([\text{GAP}(\mathbf{F}'_{P3}); \text{flatten}(\mathbf{S}); \mathbf{z}_{pose}])$$

Complexity → 3 backbones, 3 pipelines | 1 backbone, 1 pipeline, 3 heads → faster inference

Fusion → sequential | Parallel + FiLM → activity prediction is explicitly conditioned on **where** the worker's joints are, not just that a person exists

Stability → sum loss that potentially crash | kendall + smooth L1 → Dynamically balances task gradients, preventing one tiny error (Pose) from destroying all other weights (Detection, Activity)

- Make a presentation → essential problem → what is the technical problem/essential problem/issue in my implementation → keypoints enlarge the knowledge of the model (keypoints, depth, etc) → FiLM is perfect or not
- Defects in the existing method

- encounter

- Improve FiLM

1. Identify new problem → very clear problem
2. Improve the existing method
3. Write a paper

- I will try to explore all of the fundamental issues in my model
-

Scenario 1

1. application → application (using existing method) (paper 1)
2. fundamentally improvement → application (paper 2)
3. Implementation of the company (finding sponsors and real implementation)

I need filters the conference → asia only → budget limitation →

Total amount <300.000

→ i will try find the conference that focus on the application

Conference that matched my project

- **CML 2026 - Seoul, Korea** - <https://icml.cc/Conferences/2026/CallForPapers>

(July 6-12, 2026 → Deadline January 29, 2026)

- **IEEE ISIE 2026 - Nagoya, Japan** - https://attend.ieee.org/isie-2026/call-for-papers_rv/

(June 23-26, 2026 → Deadline January 31, 2026)

- **IEEE FG 2026 - Kyoto, Japan** - <https://fg2026.ieee-biometrics.org/cfp/>

(May 25-29, 2026 → Deadline January 15, 2026)

- **ACCV 2026 - Osaka, Japan** - <https://accv2026.org/>

(December 14-18, 2026 → Deadline July 5, 2026)

- **IEEE CASE 2026 - Shenyang, China** - https://hk.aconf.org/conf_226203.html

(Aug 17-21 2026 → Deadline ...)

- **IPODT 2026 - Guangzhou, China** - <https://www.ipodt.net/>

(aug 21-23 2026 → Deadline July 20 2026)

- **AMLDS 2026 - Osaka, Japan** - <https://www.amlds.site/>

(July 21-23, 2026 → Deadline February 10, 2026)

- **MLMI 2026 - Tokyo, Japan** - <https://www.mlmi.net/>

(July 17-20, 2026 → Deadline February 5, 2026)

```

validating: 100%| 6/6 [00:01<00:00, 5.35it/s, act=0.0000, det=0.9736, pose=0.0873, total=0.7181
2025-12-09 10:51:23.468 INFO Epoch 66/100 | Train: 0.6294 | Val: 1.7318 | Det: 0.8155 | Pose: 0.1214 | Act: 0.0225 | LR: 0.000267 | Weights Det:1.012 Pose:1.197 Act:1.197
Epoch 67: 100%| 26/26 [00:12<00:00, 2.06it/s, act=0.0000, det=0.7442, pose=0.1197, total=0.5176
validating: 100%| 6/6 [00:01<00:00, 5.29it/s, act=0.0000, det=1.0057, pose=0.0887, total=0.7457
2025-12-09 10:51:37.236 INFO Epoch 67/100 | Train: 0.5940 | Val: 1.9785 | Det: 0.8123 | Pose: 0.1199 | Act: 0.0030 | LR: 0.000253 | Weights Det:1.016 Pose:1.201 Act:1.202
Epoch 68: 100%| 26/26 [00:12<00:00, 2.08it/s, act=0.0000, det=0.8462, pose=0.1092, total=0.6015
validating: 100%| 6/6 [00:01<00:00, 5.35it/s, act=0.0000, det=0.9801, pose=0.0870, total=0.7110
2025-12-09 10:51:50.873 INFO Epoch 68/100 | Train: 0.5878 | Val: 1.9248 | Det: 0.8063 | Pose: 0.1206 | Act: 0.0002 | LR: 0.000240 | Weights Det:1.019 Pose:1.205 Act:1.206
Epoch 69: 100%| 26/26 [00:12<00:00, 2.06it/s, act=0.0000, det=0.7839, pose=0.1175, total=0.5411
validating: 100%| 6/6 [00:01<00:00, 5.30it/s, act=0.0000, det=0.9996, pose=0.0863, total=0.7239
2025-12-09 10:52:04.647 INFO Epoch 69/100 | Train: 0.5734 | Val: 1.7765 | Det: 0.7943 | Pose: 0.1204 | Act: 0.0123 | LR: 0.000227 | Weights Det:1.024 Pose:1.209 Act:1.210
Epoch 70: 100%| 26/26 [00:12<00:00, 2.05it/s, act=0.0000, det=0.8656, pose=0.1095, total=0.6089
validating: 100%| 6/6 [00:01<00:00, 5.38it/s, act=0.0000, det=0.9905, pose=0.1080, total=0.7345
2025-12-09 10:52:18.475 INFO Epoch 70/100 | Train: 0.5847 | Val: 1.8705 | Det: 0.7928 | Pose: 0.1187 | Act: 0.0303 | LR: 0.000214 | Weights Det:1.028 Pose:1.213 Act:1.215
2025-12-09 10:52:19.659 INFO Training curves saved: runs/workernet_unified/curves/training_curves.png
Epoch 71: 100%| 26/26 [00:12<00:00, 2.06it/s, act=0.0000, det=0.8146, pose=0.1290, total=0.5726
validating: 100%| 6/6 [00:01<00:00, 5.37it/s, act=0.0000, det=0.9950, pose=0.0845, total=0.7052
2025-12-09 10:52:33.373 INFO Epoch 71/100 | Train: 0.5356 | Val: 1.8069 | Det: 0.7851 | Pose: 0.1189 | Act: 0.0018 | LR: 0.000202 | Weights Det:1.032 Pose:1.217 Act:1.219
Epoch 72: 100%| 26/26 [00:12<00:00, 2.08it/s, act=0.0001, det=0.8434, pose=0.1012, total=0.5618
validating: 100%| 6/6 [00:01<00:00, 5.41it/s, act=0.0000, det=0.9970, pose=0.0860, total=0.7019
2025-12-09 10:52:46.975 INFO Epoch 72/100 | Train: 0.5253 | Val: 1.7839 | Det: 0.7831 | Pose: 0.1181 | Act: 0.0017 | LR: 0.000189 | Weights Det:1.036 Pose:1.220 Act:1.223

```

	precision	recall	f1-score
Checking	0.8667	0.8125	0.8387
Rotation	0.8636	0.9048	0.8837
Storing	1.0000	1.0000	1.0000
accuracy			0.8837
macro avg	0.9101	0.9058	0.9075
weighted avg	0.8838	0.8837	0.8832
Confusion Matrix:			
[[13 3 0]			
[2 19 0]			
[0 0 6]]			
FINAL ACCURACY: 88.37%			

Short Term Planning (5)

Aa Name	📅 Date	☰ Tags
<u>Untitled</u>		
<u>Write paper and submission</u>	@December 31, 2025 → January 31, 2026	
<u>Address feedback, revise code and documentation</u>		

Aa Name	📅 Date	☰ Tags
<u>Prepare figures, diagrams, and tables for paper</u>		
<u>POPW Implementation COMPLETE</u>	@December 22, 2025 → December 31, 2025	
<u>Data and result collection for paper</u>	@December 10, 2025 → December 31, 2025	
<u>Address feedback, revise code and documentation</u>		
<u>Prepare figures, diagrams, and tables for paper</u>		
<u>Address feedback, revise code and documentation</u>		
<u>Prepare figures, diagrams, and tables for paper</u>		
<u>Optimization</u>	@December 10, 2025 → December 31, 2025	
<u>Real world lab testing</u>	@December 10, 2025 → December 31, 2025	
<u>POPW protocol implementation</u>	@December 10, 2025 → December 31, 2025	
<u>Implement the spatiotemporal fingerprint</u>	@December 10, 2025 → December 31, 2025	
<u>Debugging</u>	@November 11, 2025 → December 14, 2025	
<u>Untitled</u>		
<u>Data Collection</u>	@November 11, 2025 → December 14, 2025	
<u>Finalize architecture and prepare result</u>	@October 31, 2025 → December 9, 2025	
<u>Complete workernet training & benchmarking</u>	@October 14, 2025 → December 9, 2025	
<u>Untitled</u>	@November 17, 2023 → November 30, 2023	